Effective Boolean Argument Filter

Developer Brief and Implementation Specification

Version 1.0 - Prepared for developer handoff

Purpose: turn the conceptual framework into an MVP that parses arguments, tracks yes/no transformations, detects untracked shifts, generates reactive probes, and scores effective usefulness rather than acting as a truth oracle.

1. Executive summary

This project implements a traceable argument-effect filter. It does not decide truth by authority, and it does not treat an LLM answer as the final verdict. The system evaluates whether an argument preserves its internal yes/no transformations, scope, definitions, and task effects under reactive tests.

Core principle: no untracked polarity shifts. A statement may behave like yes = no(no) only if both negations are tracked, scoped, and applied to the same object, with the same definition and context.

- The LLM may generate candidate parses, probes, and explanations, but the deterministic engine owns the verdict trace.
- Contradictions are localized and tested; the system must not use explosion logic as the main behavior.
- Every argument receives a score vector and a trace, not merely pass/fail.
- The system sorts arguments by effectiveness and structural stability; it is not a censorship or truth-assertion machine.

2. What has been established so far

2.1 Conceptual framework

- Defined truth/effect as preserved structure, not merely the presence of the word yes.
- Defined yes as no(no) only under tracked negation parity.
- Separated plain double negation from epistemic absence-of-evidence claims.
- Rejected the explosion rule as the main filter. Contradictions become test obligations, not automatic invalidation.
- Defined the system as a reactive test: an argument must respond with an expected effect under probes.
- Defined effectiveness as a score over negation consistency, scope preservation, definition stability, context fit, contradiction containment, reactive performance, and testability.

2.2 Operating analogy from the computational work

During the Xi-Jensen workflow, the working pattern evolved into a practical certification pipeline: fast candidate generation, verification, triage, deepcheck, certified merge, and iterative status reporting. This is directly useful as the architecture pattern for the argument filter.

Stage	Xi-Jensen workflow	Argument filter analogue
Generator	Fast/numpy frontier scan produced candidate labels.	LLM and parser produce candidate claim structures.
Cheap classifier	Fast labels were useful but not final.	Initial polarity/scope labels are provisional.
Verifier	Ordinary polyroots audit found mismatches and failures.	Reactive probes and deterministic checks find instability.
Triage	Mismatches and failures were isolated.	Problematic inference steps are isolated.
Deepcheck	Scaled high-precision solver became the trusted verifier.	High-strictness logical/probe evaluation becomes the trusted audit.
Certified merge	Deep labels replaced fast labels with provenance.	Verified argument labels replace provisional labels with provenance.

Latest certification status from the computational workflow: after the v2 merge there were 290 rows total, 58 deepcheck_ok rows, and 232 unverified rows. A subsequent scaled certification batch selected 25 more rows, completed 25/25 successfully, and changed 21 current certified labels. This demonstrates why the final system must preserve provenance and support iterative certification rather than relying on one fast pass.

3. Product definition

3.1 Working name

Effective Boolean Argument Filter.

3.2 Goal

Build a system that takes a claim, argument, context, and task, then checks whether the argument preserves its yes/no structure under negation, scope, definition, and reactive test effects.

3.3 Non-goals

- Do not build a truth oracle.
- Do not let an LLM produce untraceable final scores.
- Do not collapse absence of disproof into proof.
- Do not discard every argument containing a contradiction.
- Do not rely on classical explosion as the main filtering rule.

4. Core logic

4.1 Polarity states

```
type Polarity =
  | "effective_yes"
  | "effective_no"
  | "unknown"
  | "contradiction"
  | "untracked_shift"
  | "unstable";
```

4.2 Negation parity rule

For one object P, under one scope and one definition:

```
N^k(P) = P      if k is even
N^k(P) = not P  if k is odd
```

This reduction is allowed only when the invariants hold: same object, same scope, same definition, same context, and same negation type. If any invariant fails, mark the transformation as untracked_shift.

4.3 Critical distinction

Expression	Internal form	Allowed reduction
It is not the case that not P.	not(not(P))	May reduce to effective_yes if scope and object match.
There is no evidence that P is false.	not known(not(P))	Must not reduce to effective_yes.
P is not legally impossible.	not legally_impossible(P)	Must not imply physically_possible(P) without added bridge premise.
It works in simulation.	works(P, simulation)	Must not imply works(P, production) without scope bridge.

5. MVP data model

```
interface ArgumentInput {
  claim: string;
  argument: string;
  context: string;
  task: string;
  strictness: "low" | "medium" | "high";
}

interface ClaimNode {
```

```
  id: string;
  text: string;
  normalized_form: string;
  object_id: string;
  scope: string;
  context_id: string;
  definition_id: string;
  polarity: Polarity;
  confidence: number;
}

interface TransformationStep {
  from_node: string;
  to_node: string;
  transformation_type: Transformation;
  tracked: boolean;
  reason: string;
}

interface EvaluationReport {
  id: string;
  effective_polarity: Polarity;
  effectiveness_score: number;
  bogusness_score: number;
  score_vector: ScoreVector;
  trace: TransformationStep[];
  issues: Issue[];
  probes: Probe[];
  recommendation: string;
}
```

6. Parser requirements

The first-pass parser must identify claims, premises, conclusions, negations, double negations, modal terms, epistemic terms, causal claims, and conclusion markers.

Input phrase	Expected parse
not P	not(P)
not not P	not(not(P))
no evidence for P	not known(evidence_for(P))
no evidence against P	not known(not(P)) or not known(evidence_against(P))
it is false that P	false(P) or not(P), depending on context
it is not known that P	not known(P)
P does not imply Q	not(implies(P,Q))
P failed to disprove Q	failed(disprove(P,Q)); not proof(Q)

7. Contradiction handling

Do not implement contradiction -> discard. Implement contradiction -> isolate -> test dependency.

```
type ContradictionStatus =
  | "contained"
  | "conclusion_dependent"
  | "scope_resolved"
  | "temporal_resolved"
  | "definition_resolved"
  | "breaking";
```

- If the contradiction is irrelevant to the conclusion, weaken the score but do not discard.
- If the conclusion depends on the contradiction, mark contradiction_breaking.
- If the contradiction is temporal, mark temporal_resolved.

- If the contradiction is definitional, track which term version changed.

8. Reactive probe generator

The system must perturb the argument and check whether the conclusion keeps its effective polarity.

Probe type	Purpose
Invert premise	Test whether the conclusion depends on a hidden polarity assumption.
Weaken premise	Test whether the conclusion is stronger than its premises.
Remove premise	Find dependency on a premise.
Swap definition	Detect equivocation.
Ask for falsifier	Check testability.
Ask for implementation	Separate abstract truth from practical effect.
Ask for prediction	Force measurable consequences.
Ask for counterexample	Detect brittle universal claims.
Ask for dependency	Expose hidden assumptions.
Ask for measurable effect	Convert rhetoric into a task metric.

8.1 Example probe generation

```
Input argument:
  "This framework is correct because no one disproved it."

Generated probes:
  1. What would disprove it?
  2. Does the conclusion still follow if absence of disproof is removed?
  3. What concrete prediction does it make?
  4. Does it outperform a baseline?
  5. Which premise carries the proof burden?
```

9. Effectiveness scoring

```
overall_effectiveness =
  0.20 * negation_consistency
+ 0.15 * scope_preservation
+ 0.15 * definition_stability
+ 0.15 * context_fit
+ 0.10 * contradiction_containment
+ 0.15 * reactive_performance
+ 0.10 * testability
```

Bogusness increases with untracked polarity shifts, scope jumps, definition shifts, conclusions stronger than premises, missing falsifier, and failed reactive probes.

Score field	Meaning
negation_consistency	Whether yes/no transformations are tracked and parity-preserving.
scope_preservation	Whether the argument stays within the same domain/scope.
definition_stability	Whether key terms retain meaning.
context_fit	Whether the conclusion fits the declared task/context.
contradiction_containment	Whether contradictions are localized and non-explosive.
reactive_performance	How well the argument survives probes.
testability	Whether the claim exposes falsifiers or measurable effects.
implementation_relevance	Whether the claim can drive a concrete program, device, or decision.

10. System architecture

Use an LLM as an assistant, not as the authority.

```
raw argument
  -> LLM extracts structured candidate claims
  -> deterministic polarity/scope engine checks transformations
  -> LLM proposes probes
  -> deterministic scorer aggregates results
  -> report with trace and provenance
```

Component	Responsibility	Authority level
LLM parser assistant	Extract candidate claims, premises, modal terms, and possible scopes.	Advisory only
Deterministic polarity engine	Track negation parity and transformations.	Authoritative for trace
Scope/definition tracker	Detect domain, context, temporal, and definition shifts.	Authoritative for issues
Contradiction module	Localize contradiction and test conclusion dependency.	Authoritative for contradiction status
Probe generator	Create reactive tests based on detected weaknesses.	Advisory plus templates
Scoring engine	Compute explainable scores from trace and probes.	Authoritative for report
Report layer	Return JSON and human-readable explanation.	Presentation only

11. API specification

```
POST /evaluate_argument
POST /generate_probes
POST /score_probe_results
GET  /reports/{id}
```

11.1 Evaluate input

```
{
  "claim": "This method proves X",
  "argument": "There is no evidence against X, therefore X is true",
  "context": "scientific argument evaluation",
  "task": "detect whether the argument is structurally valid and effective",
  "strictness": "medium"
}
```

11.2 Evaluate output

```
{
  "id": "eval_123",
  "effective_polarity": "unstable",
  "effectiveness_score": 0.24,
  "bogusness_score": 0.76,
  "trace": [
    {
      "statement": "no evidence against X",
      "parsed_form": "not known(not X)",
      "invalid_reduction": "not not X",
      "warning": "epistemic negation was converted into ontological affirmation"
    }
  ],
  "issues": [
    "untracked polarity shift",
    "epistemic-to-ontological shift",
    "conclusion stronger than premises"
  ],
  "probes": [
    "Ask what evidence would falsify X",
    "Check whether X follows if evidence is merely absent",
    "Request concrete prediction or implementation result"
  ],
  "recommendation": "needs testing"
```

```
}
```

12. CLI MVP

```
python effective_boolean_filter.py   --claim "This method proves X"   --argument "There is no evidence against X, therefo
```

12.1 Expected CLI output

```
Effective polarity: unstable
Bogusness score: 0.76
Effectiveness score: 0.24

Main issue:
- "no evidence against X" was incorrectly treated as "not not X"

Detected structure:
- no evidence against X = not known(not X)
- not known(not X) does not imply X

Recommended tests:
1. Define what evidence would falsify X.
2. Test X in an implementation context.
3. Check whether conclusion survives if absence-of-disproof premise is removed.
```

13. Benchmark dataset requirements

Create at least 50 labeled examples. Each example must include expected issue labels, expected polarity result, and explanation.

Category	Example	Expected behavior
Clean valid case	P. Not not P. Therefore P.	effective_yes
Invalid double-negation case	No evidence that P is false. Therefore P is true.	unknown or unstable; flag epistemic shift
Scope shift case	It works in simulation. Therefore it works in production.	unstable; flag scope shift
Fallacy-fallacy case	The argument has one weak premise. Therefore the conclusion is false.	unstable; flag fallacy fallacy
Contained contradiction	The model failed in A but works in B. Therefore it is useful in restricted context B.	contradiction_contained
Bogus case	No one can disprove X, therefore X is proven.	unstable; high bogusness

14. Implementation plan

Sprint	Deliverables	Acceptance criteria
Sprint 1 - Core engine	Data schema, polarity parser, negation parity tracker, basic report output.	Can distinguish not not P from no evidence against P.
Sprint 2 - Scope and contradiction	Scope tracker, definition shift detector, contradiction containment, no-explosion behavior.	Can isolate contradictions and label scope/definition shifts.
Sprint 3 - Reactive tests	Probe generator, probe scoring, effectiveness score.	Produces at least five useful probes tied to detected weaknesses.
Sprint 4 - API and UI	API endpoints, JSON reports, simple dashboard.	Reports are traceable and reproducible.
Sprint 5 - Benchmarks	50+ examples, scoring calibration, regression tests.	CI catches regression on key examples.

15. Suggested repository structure

```
effective-boolean-filter/
  README.md
  pyproject.toml or package.json
```

```
src/
  schema/
  parser/
  polarity/
  scope/
  contradiction/
  probes/
  scoring/
  api/
  report/
tests/
  fixtures/
  test_negation_parity.*
  test_scope_shifts.*
  test_contradiction_containment.*
  test_probe_generation.*
benchmarks/
  examples.jsonl
docs/
  developer_brief.pdf
  developer_brief.docx
```

16. Initial acceptance tests

Test	Input	Required output
Double negation	It is not the case that not P.	effective_yes if invariants hold.
Triple negation	It is not the case that it is not the case that not P.	effective_no if invariants hold.
Epistemic non-proof	No evidence that P is false; therefore P.	untracked_shift or unstable.
Legal/physical shift	Not legally impossible; therefore physically possible.	scope_shift.
Simulation/production shift	Works in simulation; therefore works in production.	scope_shift and needs implementation probe.
Contained contradiction	Failed in A, works in B, therefore useful in B.	contradiction_contained.
Explosion prevention	A and not A are present.	Do not infer arbitrary B; isolate contradiction.

17. Engineering risks and guardrails

- LLM overreach: require structured JSON and deterministic validation.
- Score opacity: every score must include reason strings and trace links.
- False certainty: output effective polarity and confidence, not absolute truth.
- Over-penalizing contradiction: use containment and dependency checks.
- Undetected definition shifts: track definition_id and term versions explicitly.
- Prompt injection in arguments: treat the argument text as data, never as instructions to the system.
- Benchmark gaming: keep adversarial examples and regression tests.

18. Immediate next steps for the developer

1. Create the repository and add schema types for ArgumentInput, ClaimNode, TransformationStep, ScoreVector, Probe, and EvaluationReport.

2. Implement the deterministic negation parity module first. Do not integrate the LLM until this module passes tests.

3. Build the parser for a controlled phrase set: not P, not not P, no evidence for P, no evidence against P, not known P, false that P.

4. Implement scope metadata and definition IDs.

5. Implement the contradiction containment module without explosion.

6. Add a CLI that returns JSON and a human-readable report.

7. Add the initial benchmark examples and regression tests.

8. Only after the deterministic core works, add an LLM wrapper for extraction and probe generation.

Appendix A - Developer instruction summary

Build the system as a traceable argument-effect filter, not as a truth oracle.

```
Every yes/no transformation must be tracked.
Every contradiction must be localized.
Every conclusion must preserve scope.
Every accepted claim must survive reactive probes.
```

MVP success condition:

```
not not P -> effective_yes

but

no evidence against P -> unknown, not effective_yes
```

Appendix B - Development pattern learned from the certification work

The certification pipeline demonstrated that a fast first pass is useful, but final authority should come from a higher-reliability verifier with provenance. The same pattern should be used here.

```
candidate generation
  -> fast structural filter
  -> verifier/probe queue
  -> triage of mismatches and failures
  -> deepcheck of hard cases
  -> certified merge with provenance
  -> status report and next-batch planning
```